

TDs #12 et #13 - mini-projet : Arène des Personnages

Esma TALHI & Alan ZIREK

Objectif du mini-projet

Ce mini-projet consiste à créer un petit jeu de combat dans lequel différents personnages s'affrontent dans une arène. Chaque personnage possède des caractéristiques propres, utilise des pouvoirs spécifiques pour attaquer, et les vainqueurs récupèrent des artefacts aux perdants. Les duels suivent des règles simples et leurs résultats sont enregistrés dans un fichier texte.

L'objectif est de construire ce système progressivement, en plusieurs étapes, afin de mobiliser les notions de classes, héritage, redéfinition de méthodes, listes d'objets et gestion simple de fichiers.

Pouvoirs et personnages

1. Classe Pouvoir

1. Créez une classe `Pouvoir` représentant un pouvoir simple.

Un pouvoir sera caractérisé par :

- un attribut `nom` (chaîne de caractères) ;
- un attribut `degats` (entier).

2. Ajoutez un constructeur initialisant ces deux attributs.

3. Ajoutez une méthode `__str__()` qui renvoie une représentation lisible du pouvoir, par exemple : `"Pouvoir : Flamme (20)"`.

2. Classe Personnage

4. Créez une classe `Personnage` qui servira de classe mère pour tous les personnages du jeu.

Un personnage sera caractérisé par :

- un `nom` (public) ;
- une `_categorie` (protégée) ;
- des points de vie actuels `_pv` (protégés) ;

- des points de vie maximum `_max_pv` ;
- une liste de pouvoirs `pouvoirs` (liste d'objets `Pouvoir`) ;
- une liste d'artefacts `artefacts` (comme par exemple Pierre magique, Livre de sorts, etc.).

5. Dans le constructeur de `Personnage`, initialisez :

- `_pv` et `_max_pv` avec la même valeur ;
- la liste `pouvoirs` à partir d'un paramètre optionnel (par défaut liste vide) ;
- la liste `artefacts` avec un élément initial, par exemple : `"Artefact initial"`.

6. Ajoutez les méthodes suivantes dans `Personnage` :

- `est_vivant()` : renvoie `True` si le personnage a plus de 0 PV ;
- `recevoir_degats(dmg)` : réduit `_pv` sans descendre en dessous de 0 ;
- `soigner(pts)` : augmente `_pv` sans dépasser `_max_pv` ;
- `attaquer(cible)` :
 - si la liste `pouvoirs` n'est pas vide, choisit un pouvoir aléatoire et applique ses dégâts ;
 - sinon, inflige des dégâts faibles (par exemple 5). La méthode `attaquer` doit renvoyer un string contenant des informations de l'attaque, par exemple `Merlin (Mage) lance un sort sur Valkyrie (23) -- PV Valkyrie : 221`.
- `prendre_artefact(perdant)` : implémente la règle d'artefact :
 - si le perdant possède au moins un artefact, le vainqueur prend le premier artefact de la liste ;
 - sinon, un message indique que le perdant n'avait plus d'artefact.
- `__str__()` : renvoie une description complète du personnage (nom, catégorie, PV, artefacts).

¹Toutes les méthodes `attaquer` dans toutes les classes doivent renvoyer un texte ressemblant à celui-là. Vous avez aussi la possibilité de personnaliser le texte

Héritage : types de personnages

3. Classes dérivées

7. Créez les classes suivantes, héritant de `Personnage` :

- `Gobelin` ;
- `Chevalier` ;
- `Archer` ;
- `Mage` ;
- `Sage`.

8. Dans chaque classe, définissez un constructeur appelant `super().__init__()` avec :

- un nombre de PV adapté (par exemple : `Gobelin` = 50, `Chevalier` = 120, `Archer` = 70, `Mage` = 60, `Sage` = 80) ;
- la catégorie correspondante ("`Gobelin`", "`Chevalier`", etc.) ;
- une liste de pouvoirs éventuellement vide ou fournie en paramètre.

9. Règles spécifiques :

- **`Chevalier`** : ajoute un attribut `_resistance` (par exemple 0.10) et redéfinit `recevoir_degats` pour réduire les dégâts subis (Il réduit les dommages reçus de 10%) ;
- **`Mage`** :
 - possède en plus un attribut `puissance_magie` (par exemple 20) ;
 - redéfinit `attaquer(cible)` pour lancer un sort dont les dégâts varient autour de `puissance_magie`. Quand un `Mage` attaque, son sort perd ou gagne en puissance avant d'atteindre la cible. Il faut donc générer une valeur aléatoire autour de sa `puissance_magie`. Si sa puissance a été initialisée 20, son sort sera une valeur entre 16 et 22 par exemple ;
- **`Sage`** :
 - possède un attribut `pouvoir` (dégâts fixes) ;
 - possède un attribut `potion` (quantité de PV gagnée) ;
 - ajoute une méthode `boire_potion()` qui augmente temporairement ses PV ;
 - redéfinit `attaquer(cible)` pour utiliser son pouvoir. Lorsqu'un `Sage` attaque, il inflige les dégâts de son pouvoir principal, et gagne en même temps +4 % de puissance sur ce même pouvoir. (Exemple : pouvoir initial : 20 dégâts, près un coup : pouvoir = 20.8 → arrondi à 20, après quelques coups : 20 → 21 → 22 → ...)

Duel et règles de combat

4. Fonction `duel(p1, p2)`

10. Écrivez une fonction globale :

```
def duel(p1, p2):
```

```
    ...
```

Cette fonction doit :

- créer une liste `historique` qui contiendra toutes les actions du combat ;
- ajouter au début un message du type : `"--- Début du combat : p1 vs p2 --- "`, remplacer dans l'affichage `p1`, `p2` par les noms des personnages;
- exécuter une boucle de combat :
 1. `p1` attaque `p2` et le message renvoyé par `attaquer()` est ajouté à `historique` ;
 2. si `p2` n'est plus vivant, la boucle s'arrête (utiliser `break`);
 3. `p2` attaque `p1`, et ainsi de suite.

11. À la fin du duel :

- si seul `p1` est vivant, le vainqueur est `p1` et le perdant est `p2` ;
- si seul `p2` est vivant, le vainqueur est `p2` et le perdant est `p1` ;
- si les deux personnages tombent à 0 PV (double KO), il n'y a pas de vainqueur.

12. Si un vainqueur existe :

- appelez `vainqueur.prendre_artefact(perdant)` ;
- ajoutez le message renvoyé à la liste `historique` ;
- ajoutez un message final : `"Vainqueur : NOM"`.

13. La fonction `duel` doit enfin renvoyer un couple (`vainqueur`, `historique`).

Sauvegarde dans un fichier texte

5. Fonction `sauvegarde_resultat`

14. Écrivez une fonction :

```
def sauvegarde_resultat(texte):
```

```
    ...
```

Cette fonction doit ouvrir (ou créer) un fichier texte nommé `resultats.txt` en mode ajout, puis écrire une ligne contenant :

- la date et l'heure courantes ;
- une description du duel et du vainqueur.

Par exemple :

```
2025-11-29 14:12:03 - Gor vs Arthur -> Arthur gagne un artefact
```

Programme principal

6. Fonction `main()`

15. Dans une fonction `main()` :

- créez quelques pouvoirs (par exemple "Flamme", "Vent", "Ombre", "Sagesse") ;
- créez un Gobelin, un Chevalier, un Archer, un Mage et un Sage avec des pouvoirs adaptés ;
- affichez l'état initial de chaque personnage (en utilisant `print(personnage)`).

16. Créez une liste de duels, par exemple :

```
combats = [(gob, chev), (arch, mage), (sage, chev)]
```

17. Pour chaque couple (p1, p2) de la liste :

- appelez `duel(p1, p2)` ;
- affichez toutes les lignes de l'historique ;
- appelez `sauvegarde_resultat(...)` avec un résumé du résultat.

Remarque

Ce mini-projet pourra servir de base à des extensions : ajout de nouveaux types de personnages, nouveaux pouvoirs, règles supplémentaires (tournois, équipes, etc.). Dans un premier temps, concentrez-vous sur la bonne conception des classes, la clarté du code et le respect des règles de jeu décrites.